2.1. Estándares web: origen y evolución de la Web

De los primeros navegadores a la guerra de navegadores de los años 90, el nacimiento de HTML y HTTP, y por qué el W3C empezó a hablar de estándares web.

Para entender por qué HTML y el resto de tecnologías web son como son, conviene conocer primero cómo y por qué nacieron. La historia de la Web está marcada por la necesidad de intercambiar información de forma sencilla, y por una intensa competencia comercial entre los primeros navegadores.

El nacimiento de la Web (1989-1990)

En 1989, Tim Berners-Lee, investigador del CERN, propuso un sistema de gestión de información basado en hipertexto para facilitar el intercambio de documentos entre científicos. Junto con Robert Cailliau, desarrolló en 1990 los tres pilares que siguen sosteniendo la Web actual: el lenguaje HTML para estructurar documentos, el protocolo HTTP para transferirlos, y el primer navegador, capaz de mostrarlos.

¿Por qué un lenguaje tan simple como HTML?

En los primeros momentos de internet, los escasos usuarios empleaban lenguajes como TeX y PostScript para representar documentos, pero resultaban demasiado complejos, teniendo en cuenta que debían poder leerse en todo tipo de computadoras. Para llegar a un mayor número de equipos, y por tanto de personas usuarias, se optó por idear un lenguaje de intercambio más simple (HTML) y un protocolo de red igualmente sencillo (HTTP).

```
%!
% Ejemplo de código PostScript
  /Courier findfont
  20 scalefont
% Courier tamaño 20
  newpath
  72 72 moveto
% Cursor en la esquina
% inferior izquierda (72,72)
  (¡Hola mundo!) show
  showpage
```

Frente a la complejidad de un lenguaje como el anterior, HTML se presenta como una aplicación de SGML (*Standard Generalized Markup Language*), un lenguaje mucho más general para definir el formato de documentos, del que también deriva XML. El nombre de HTML resume bien su propósito:

HyperText	Texto con enlaces a otros documentos.

Mark-up	Lenguaje de marcas, basado en etiquetas.
Language	Un lenguaje con su propia sintaxis y reglas.

Los primeros navegadores

El primer navegador fue creado por el propio Tim Berners-Lee en el CERN, en 1990. Se llamó WorldWideWeb y, aunque era sofisticado para su época, solo funcionaba en estaciones NeXT. Poco después apareció Mosaic, que funcionaba inicialmente en sistemas Unix y se popularizó gracias a que el National Center for Supercomputing Applications (NCSA) preparó versiones para Windows y Macintosh.

La batalla de los navegadores

Poco después entró en el mercado Netscape y, con algo más de retraso, Microsoft Internet Explorer. Comenzó entonces una competencia muy intensa por hacerse con el mayor número de personas usuarias posible:

- En 1995, Netscape era considerado el estándar de facto para los navegadores web, con solo algunos competidores.
- Ese mismo año, Microsoft, consciente del éxito de Netscape, lanzó Windows 95 incorporando la primera versión de Internet Explorer.
- Más tarde, en su segunda versión, Microsoft liberó su navegador del sistema operativo, permitiendo que funcionara de forma independiente a Windows. A la vez, puso en marcha estrategias comerciales para ganar mercado, como incluir su propia página web como opción de inicio predeterminada.

De las recomendaciones a los estándares

Sin el apoyo de organismos y empresas, el W3C (*World Wide Web Consortium*) llamaba *recomendaciones* a una serie de normas básicas que permitían a quienes creaban contenidos y diseñaban páginas trabajar sin preocuparse de en qué sistema o navegador se iba a visualizar el resultado. Desde 1998, estas recomendaciones pasaron a denominarse estándares.

A efectos prácticos, diseñar con estándares supone aplicar esas normas básicas con independencia de la metodología de trabajo que se utilice, garantizando que el resultado se muestre de forma consistente en cualquier navegador que los respete.

2.2. La Web responsive y el enfoque Mobile First

Por qué la llegada de los smartphones obligó a que la Web se volviera flexible, qué son las media queries y en qué consiste diseñar pensando primero en el móvil.

En 2007, Apple introdujo en el mercado su primer iPhone. A partir de ese momento, los teléfonos móviles empezaron a considerarse verdaderos dispositivos de acceso a la Web, y no un simple complemento del ordenador de sobremesa.

Los diseños web tuvieron que empezar a dar respuesta a pantallas nuevas: de menor resolución y tamaño, y con proporciones distintas a las habituales hasta entonces. La Web necesitaba volverse flexible.

Media queries

Aunque la tecnología o el concepto de *media query* es casi tan antiguo como la propia Web, es a partir de este momento cuando su desarrollo adquiere una importancia especial. Media Queries es un módulo de CSS3 que permite adaptar la representación del contenido a las características del dispositivo, como su resolución de pantalla.

¿En qué consiste el Mobile First?

Mobile First es un movimiento o tendencia de diseño que pone en primer lugar a los dispositivos móviles. A la hora de elaborar una página web, un contenido o cualquier tipo de software, se tiene siempre en mente, en primer lugar, a los dispositivos *smart* como tablets o teléfonos móviles, en lugar de la tradicional pantalla de ordenador.

Mobile First frente a Web responsive

Aunque están relacionados, no son exactamente lo mismo:

- La Web responsive ofrece diferentes opciones o alternativas de diseño según el dispositivo desde el que se accede, adaptando el mismo contenido a cada tamaño de pantalla.
- Mobile First prioriza directamente el diseño para *smartphones*, dando por sentado que lo que funciona bien en estos dispositivos también lo hará en pantallas de mayor tamaño.

Mobile First sostiene, además, que ofrecer una buena experiencia de uso en móvil es rentable, ya que los buscadores premian un buen diseño para estos dispositivos a la hora de posicionar los resultados de búsqueda.

2.3. HTML5: qué aporta y sintaxis básica de las etiquetas

Las novedades que trajo HTML5 respecto a versiones anteriores y la sintaxis básica de una etiqueta: apertura, cierre, atributos y contenido.

HTML5 es la última versión de HTML, aunque el término representa en realidad dos conceptos diferentes:

- Un conjunto mucho más amplio de tecnologías que permite que los sitios web y las aplicaciones sean más diversos y de mayor alcance. A este conjunto se le suele llamar "HTML5 y amigos" (HTML, CSS, JavaScript), a menudo reducido simplemente a "HTML5".

- Una nueva versión concreta de HTML, con nuevos elementos, atributos y comportamientos respecto a las versiones anteriores.

Qué aporta HTML5

Más allá de nuevas etiquetas, HTML5 introdujo mejoras importantes en varios frentes:

- Semántica: permite describir con mayor precisión cuál es el contenido de cada parte del documento.
- Conectividad: permite comunicarse con el servidor de formas nuevas e innovadoras.
- Sin conexión y almacenamiento: permite que las páginas web almacenen datos localmente en el lado del cliente y operen sin conexión de forma más eficiente.
- Multimedia: ofrece un excelente soporte nativo para contenido de audio y vídeo.
- Gráficos y efectos 2D/3D: incorpora nuevas características para trabajar con gráficos en la web, como `canvas` 2D, WebGL o SVG.
- Rendimiento e integración: proporciona una mayor optimización de la velocidad y un mejor aprovechamiento del hardware.
- Acceso al dispositivo: ofrece APIs para usar diversos componentes de entrada y salida del dispositivo.
- CSS3: se acompaña de una nueva generación de CSS, con muchas más opciones para crear diseños sofisticados.

Sintaxis básica de una etiqueta

HTML5, igual que XHTML o XML, se basa en etiquetas o marcas que, en la mayoría de los casos, constan de una apertura (`<etiqueta>`) y un cierre (`</etiqueta>`). El texto comprendido entre ambas etiquetas forma parte del elemento estructural que define esa etiqueta:

```
<p>Esto es un párrafo</p>
```

Una etiqueta puede además incluir atributos, que aportan información adicional sobre el elemento en forma de pares *atributo="valor"*:

```
<html lang="es"> … </html>
```

En este ejemplo, `html` es el nombre del elemento, `lang` es un atributo, `"es"` es su valor, y todo lo que queda entre la apertura y el cierre es el contenido del elemento.

2.4. Estructura del documento: DOCTYPE, head, body, metadatos y viewport

El DOCTYPE y su función, la división entre head y body, los metadatos del documento y la etiqueta meta viewport para controlar la ventana gráfica.

El DOCTYPE

A través del DOCTYPE (declaración de tipo de documento) se hace referencia a la DTD (*Document Type Definition*) que describe las reglas del documento. El DOCTYPE informa al navegador sobre qué versión de HTML se está usando; se trata de una declaración, no de un elemento ni de una etiqueta. En HTML5, el DOCTYPE que se usa es muy sencillo:

```
<!DOCTYPE html>
```

Existen otros DOCTYPE, propios de versiones anteriores de HTML y de XHTML, que todavía pueden verse en algunos sitios, aunque su tendencia es a desaparecer en favor del sencillo DOCTYPE de HTML5.

Head y body

`<head>` (cabecera) y `<body>` (cuerpo) son la primera gran división de un documento HTML. La información contenida en `head` no es visible en la pantalla del navegador o *viewport*, al contrario de lo que ocurre con la información contenida en `body`.

Metadatos

Los metadatos contienen información sobre el propio documento, como por ejemplo la autoría, la descripción para los buscadores, el juego de caracteres o las palabras clave. Un lenguaje de marcado como HTML integra metadatos y marcas para aportar otras características estructurales o semánticas dentro del contenido. Podemos distinguir varios tipos de metadatos:

Descriptivos	Sirven para conocer o interpretar una fuente de información.
Técnicos	Sirven para decodificar y representar archivos.
De preservación	Se emplean para la gestión a largo plazo de archivos.
De derechos	Informan sobre los derechos de propiedad intelectual.
Estructurales	Describen las relaciones entre las partes de un recurso.

El viewport

La ventana gráfica o *viewport* es el área visible de la página web para la persona usuaria. Su tamaño varía según el dispositivo, y será mucho más pequeño en un teléfono móvil que en una tablet o en un ordenador personal.

Con HTML5 se introdujo un método que permite a quienes diseñan páginas web tomar el control de la ventana gráfica, a través de la etiqueta `<meta>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

La parte `initial-scale=1.0` establece el nivel de zoom inicial cuando el navegador carga la página por primera vez, mientras que `width=device-width` indica que el ancho de la página debe coincidir con el ancho de la pantalla del dispositivo. Sin esta etiqueta, muchos navegadores móviles mostrarían la página como si se tratara de una pantalla de escritorio, obligando a hacer zoom para leerla.

2.5. Elementos de texto y listas en HTML5

Etiquetas para dar significado a fragmentos de texto (small, cite, code, q, blockquote...) y los distintos tipos de listas: ul, ol y dl.

Repasemos algunos de los elementos HTML5 más habituales para dar formato semántico a fragmentos de texto, y las distintas formas de crear listados de elementos.

Otras etiquetas de texto en HTML5

ETIQUETA	SIGNIFICADO
`<small>`	Define texto más pequeño, como notas a pie de página y otros comentarios.
`<cite>`	Define el título de una obra, como un libro o una pintura.
`<address>`	Información de contacto.
`<code>`	Define un fragmento de código.
`<data>`	Añade una traducción legible por máquina de un contenido dado.
`<dfn>`	Especifica un término que se va a definir dentro del contenido.
`<del>`	Define un texto que ha sido eliminado.

ETIQUETA	SIGNIFICADO
`<ins>`	Define un texto que ha sido insertado.
`<details>`	Especifica detalles adicionales que la persona usuaria puede mostrar u ocultar.
`<q>`	Define una cita corta.
`<blockquote>`	Define una cita larga y en bloque.

Listados

Es posible crear listas de elementos en HTML con las siguientes etiquetas:

`<ul>`	Lista desordenada.
`<ol>`	Lista ordenada (numerada).
`<li>`	Cada uno de los elementos de la lista.

Las listas se pueden anidar unas dentro de otras: un elemento `<li>` puede contener a su vez una lista completa, como en el siguiente ejemplo:

```
<ul>
  <li>
    <ul>
       <li>
         <ol>
            <li>...</li>
         </ol>
       </li>
    </ul>
  </li>
</ul>
```

Listas de definición

Además de las listas ordenadas y desordenadas, existe la lista de definición, pensada para presentar pares de término y descripción:

`<dl>`	Contenedor de la lista de definición.
`<dt>`	El término que se define.
`<dd>`	La descripción de ese término.

Aunque existen atributos HTML que permiten cierta personalización de los listados, este método no es el más adecuado ni el más eficaz. Más adelante veremos que CSS dispone de propiedades que permiten mejorar su aspecto, e incluso incluir imágenes propias para las viñetas.

2.6. Enlaces e imágenes

La etiqueta a y sus tipos de destino, y la etiqueta img con sus atributos src y alt para incorporar imágenes a una página web.

Enlaces: la etiqueta <a>

La etiqueta `<a>` determina un hipervínculo. Se utiliza para enlazar de una página a otra, y tiene un atributo especialmente importante: `href`, que indica el destino del enlace. Una página vinculada se muestra normalmente en la ventana actual del navegador, a menos que se especifique otro destino.

```
<section>
    <p>
        El <a href="https://es.wikipedia.org/wiki/Hipertexto" target="_blank">hipertexto</a>
        es una herramienta con estructura no secuencial que permite crear, agregar, enlazar
        y compartir información de diversas fuentes por medio de enlaces asociativos.
    </p>
</section>
```

Los vínculos se pueden dividir en varios tipos según hacia dónde se dirigen. La diferencia está en el valor que se incluya en su atributo `href`; por ejemplo, un enlace extradominio busca una URL en internet o en una intranet, por lo que se debe escribir la URL completa, incluyendo el protocolo (`https://...`).

Imágenes: la etiqueta <img>

Con la etiqueta `<img>` se pueden vincular imágenes en una página web. La imagen debe estar en alguno de los formatos soportados por el navegador y ubicada en un directorio o URL accesible.

La etiqueta `<img>` tiene dos atributos obligatorios:

`src`	Especifica la ruta hasta la imagen.
`alt`	Especifica un texto alternativo para el caso de que la imagen no pueda verse por cualquier motivo (un error de carga, una conexión lenta, o porque la persona usuaria navega con un lector de pantalla). Este atributo es además fundamental para la accesibilidad del documento.

```
<img src="img/globos.jpg" alt="Dos globos aerostáticos">
```


globos aerostáticos de colores volando sobre un valle al amanecer

A diferencia de la mayoría de etiquetas HTML, `<img>` es un elemento vacío: no envuelve contenido ni necesita etiqueta de cierre.

2.7. Audio y vídeo en HTML5

Las etiquetas video y audio, los formatos y códecs más habituales, su compatibilidad entre navegadores y el uso de source para ofrecer alternativas.

Vídeo

Incorporar vídeo en una página web es, en la actualidad, muy fácil gracias a la etiqueta `<video>`. Los formatos de vídeo más comunes hoy en día son OGG, MP4 y WebM, cada uno con su propio códec: OGG usa el códec Theora, MP4 usa H.264 y WebM usa VP8.

No todos los navegadores son compatibles con todos los formatos, por lo que conviene conocer la tabla de compatibilidades:

Navegador ⟷	MP4	WebM	OGG
Edge ⟷	Sí	Sí	Sí
Chrome ⟷	Sí	Sí	Sí
Firefox ⟷	Sí	Sí	Sí

Navegador ⟷	MP4	WebM	OGG
Safari ⟷	Sí	Sí	No
Opera ⟷	Sí	Sí	Sí

Si queremos asegurarnos la correcta visualización de nuestros vídeos en cualquier navegador, podemos incluir el elemento `<source>`, con su atributo `src`, dentro de `<video>`. Esto permite ofrecer diferentes formatos de un mismo vídeo, de manera que el navegador elija el primero que sea capaz de reproducir:

```
<video controls width="480">
    <source src="pelicula.mp4" type="video/mp4">
    <source src="pelicula.webm" type="video/webm">
    <source src="pelicula.ogg" type="video/ogg">
    Tu navegador no admite la reproducción de este vídeo.
</video>
```

Audio

HTML5 también ofrece la posibilidad de reproducir audio, de forma muy similar a como se incorpora el vídeo. La etiqueta que se debe usar en este caso es `<audio>`. Las opciones de formato más comunes son OGG (códec Vorbis), MP3 (códec AAC) y WAV (códec ACM).

Navegador ⟷	MP3	WAV	OGG
Edge / IE ⟷	Sí	Sí	Sí
Chrome ⟷	Sí	Sí	Sí
Firefox ⟷	Sí	Sí	Sí
Safari ⟷	Sí	Sí	No
Opera ⟷	Sí	Sí	Sí

Igual que ocurre con `<video>`, para garantizar que el audio se escuche correctamente en cualquier navegador conviene incluir varias alternativas mediante el elemento `<source>` dentro de `<audio>`:

```
<audio controls>
    <source src="cancion.mp3" type="audio/mpeg">
    <source src="cancion.ogg" type="audio/ogg">
    Tu navegador no admite la reproducción de este audio.
</audio>
```

2.8. Marcos con iframe y tablas

Cómo incrustar contenido de otras páginas con iframe y cómo construir tablas de datos con table, tr, th y td.

La etiqueta <iframe>

Es posible incluir elementos de otras páginas a través de un "marco", usando la etiqueta `<iframe>`. Esto permite insertar, por ejemplo, mapas de Google, otras herramientas externas, o vídeos de plataformas de *streaming* como YouTube o Vimeo. El término inglés *streaming* (también llamado transmisión, transmisión por secuencias, lectura en continuo o descarga continua) se refiere a una retransmisión digital.

```
<iframe
    src="https://www.google.com/maps/d/embed?mid=1HUy_deaGFFGVZX28B7bhbgC9h3U&hl=es"
    width="600" height="450" frameborder="0" style="border:0" allowfullscreen>
</iframe>
```

Tablas

Para crear una tabla en HTML se usa la etiqueta `<table>`. Una tabla HTML consta de un elemento `<table>` y uno o más elementos `<tr>`, `<th>` y `<td>`:

`<tr>`	Define una fila de la tabla.
`<th>`	Define una celda de encabezado.
`<td>`	Define una celda de datos.

Existen otros elementos que pueden incluirse en una tabla, como `<caption>` (título de la tabla), `<colgroup>` (agrupación de columnas), `<thead>`, `<tfoot>` y `<tbody>` (agrupación de la cabecera, el pie y el cuerpo de la tabla).

```
<table>
    <tr>
        <th colspan="2">Largometrajes</th><th>Año</th>
    </tr>
    <tr>
        <td>Ciudadano Kane</td><td>Director, Actor, Guionista</td><td>1941</td>
    </tr>
    <tr>
        <td>El tercer hombre</td><td>Actor</td><td>1949</td>
    </tr>
    <tr>
        <td>Sed de mal</td><td>Director, Actor, Guionista</td><td>1957</td>
    </tr>
</table>
```

El atributo `colspan`, usado en el ejemplo sobre la primera celda de encabezado, hace que esa celda ocupe el espacio de dos columnas; su equivalente para filas es el atributo `rowspan`.

2.9. Elementos semánticos y clasificación en bloque o en línea

Las etiquetas semánticas que introdujo HTML5 (header, nav, main, article...) y la diferencia entre elementos de bloque y elementos en línea.

Elementos semánticos de HTML5

Antes de HTML5, la estructura de una página se construía casi siempre a base de elementos `<div>` genéricos, identificados mediante atributos `id` o `class` (por ejemplo `<div id="header">` o `<div class="nav">`). El navegador no tenía forma de saber que ese `div` concreto era, en realidad, la cabecera o el menú de navegación de la página.

HTML5 introdujo elementos semánticos que resuelven este problema, describiendo con su propio nombre el papel que cumplen dentro del documento:

ELEMENTO	SIGNIFICADO
`<header>`	Cabecera de la página o de una sección.
`<nav>`	Bloque de enlaces de navegación.

ELEMENTO	SIGNIFICADO
`<main>`	Contenido principal del documento, único por página.
`<article>`	Contenido independiente y autoconclusivo (una noticia, un post).
`<section>`	Sección temática dentro de un documento o artículo.
`<aside>`	Contenido relacionado pero secundario, como una barra lateral.
`<footer>`	Pie de página o de una sección.

Usar estos elementos en lugar de `<div>` genéricos aporta ventajas reales: mejora la accesibilidad (los lectores de pantalla pueden anunciar "cabecera" o "navegación" en lugar de un simple "grupo"), facilita el posicionamiento en buscadores (SEO), y hace que el propio código HTML sea más fácil de leer y mantener, ya que su estructura describe por sí misma la organización de la página.

Elementos en bloque y elementos en línea

En general, los elementos HTML se pueden clasificar en dos categorías principales, según cómo se presentan en un contexto de estilos por defecto (las hojas de estilo propias del navegador):

- Elementos en bloque (*block-level elements*): ocupan todo el ancho disponible del contenedor y se colocan uno debajo del otro en el flujo del documento. Algunos ejemplos comunes son `<div>`, `<p>`, `<h1>` a `<h6>`, `<ul>`, `<ol>`, `<li>`, `<section>` y `<article>`.
- Elementos en línea (*inline elements*): ocupan solo el espacio necesario dentro del flujo del texto y no provocan saltos de línea. A menudo se colocan uno al lado de otro en la misma línea. Algunos ejemplos son `<span>`, `<a>`, `<strong>`, `<em>`, `<i>`, `<b>`, `<img>`, `<input>` y `<button>`.

Es importante tener en cuenta que esta clasificación puede variar según los estilos aplicados: elementos como `<div>`, `<p>` o `<span>` pueden cambiar su comportamiento de bloque a en línea, y viceversa, mediante la propiedad CSS `display`:

```
<p style="display:inline">Esto es un párrafo</p>
```

Es recomendable utilizar las herramientas de desarrollo o inspección de elementos del navegador para examinar el comportamiento y las propiedades aplicadas a los elementos de una página concreta; esto ayuda a comprender mejor cómo se presentan y cómo se comportan realmente en cada contexto.

2.10. Formularios HTML: estructura y controles básicos

La etiqueta form, el elemento input y sus valores de tipo más comunes, la diferencia entre id y name, y el uso correcto de label.

Los formularios son herramientas fundamentales para la interacción de las personas usuarias a través de las páginas web. Los encontramos en prácticamente todas las páginas que visitamos a diario: a veces de forma clara y explícita, como en las secciones habituales "Contacta con nosotros" de una web institucional o corporativa; otras veces, sin ser tan conscientes de su existencia, como ocurre con el campo para añadir un comentario de una red social como Instagram, que no deja de ser un pequeño formulario con un solo campo de texto.

Añade un comentario... Usa un pequeño formulario HTML

dro para añadir un comentario en una publicación de Instagram

La etiqueta <form>

Con la siguiente estructura se crea un formulario en HTML:

```
<form action="gestiona_datos.php" method="post">
    …
</form>
```

La etiqueta `<form>` y el resto de elementos del formulario pueden contener una serie de atributos propios, además de otros comunes a cualquier etiqueta HTML, como `type`, `id`, `name`, `class` o `alt`. Entre los atributos propios de `<form>` destacan:

`action`	La URL o el script que procesará los datos enviados.
`method`	El método HTTP utilizado para el envío, normalmente `get` o `post`.

El elemento <input>

El elemento `<input>` es un campo de entrada donde se pueden introducir datos. Dependiendo del valor de su atributo `type`, `<input>` mostrará una apariencia diferente y admitirá una forma de introducir datos, y un tipo de dato, distinto.

Un caso habitual es el atributo `type` con valor `radio` (botón de opción) o `checkbox` (casilla de verificación). La diferencia entre ambos es que `radio` solo permite seleccionar una opción de un grupo, mientras que `checkbox` permite elegir una o varias opciones. Ambos pueden llevar además el atributo `checked`, que los marca como seleccionados por defecto.

```
<p>¿Cuál es tu color favorito?</p>
<input type="radio" id="rojo" name="color" value="rojo">
```

```
<label for="rojo">Rojo</label>
<input type="radio" id="azul" name="color" value="azul" checked>
<label for="azul">Azul</label>
```

En el ejemplo anterior, cada campo incluye un atributo `id` y otro `name`. La diferencia entre uno y otro es que `id` es un identificador único para cualquier etiqueta HTML de la página, mientras que `name` es un identificador, no necesariamente único, que se usa para los elementos de formulario y que determina con qué nombre se enviará ese dato al servidor.

La etiqueta <label>

`<label>` es una etiqueta que se aplica para enunciar algún elemento de un formulario, como `input`, `select` o `textarea`. Puede incluirse antes o después del elemento etiquetado, siempre que su atributo `for` tenga el valor correcto, es decir, que coincida con el `id` del campo al que se refiere, como se ha visto en el ejemplo anterior.

Usar correctamente `<label>` no es solo una cuestión estética: también amplía la zona sobre la que se puede pulsar para activar el campo (al hacer clic sobre la etiqueta se activa también el campo asociado), y resulta fundamental para que las personas que navegan con lectores de pantalla sepan qué información se espera en cada campo.

2.11. Formularios HTML: tipos de campo y atributos avanzados

Campos especiales como color, file o datalist, y atributos que mejoran la validación y la experiencia de uso: pattern, placeholder, autofocus, required y autocomplete.

Además de los tipos de campo más habituales, el atributo `type` de `<input>` admite muchos otros valores, y existen atributos adicionales que mejoran tanto la validación como la experiencia de la persona usuaria.

text, password y hidden

`text` es el valor por defecto: si no se indica ningún `type`, el campo se mostrará como un campo de texto normal. En el caso de `password`, su contenido no se muestra como texto legible, para aportar confidencialidad, sino con un carácter único (normalmente un punto o un asterisco). El valor `hidden` convierte al campo en invisible en el navegador, aunque sus datos se envían igual que los demás; suele usarse para recoger información no relevante para la persona usuaria, pero sí para la aplicación.

```
<label for="usuario">Usuario:</label>
<input type="text" id="usuario" name="usuario">
<label for="clave">Clave:</label>
<input type="password" id="clave" name="clave">
<input type="hidden" name="idioma" value="es">
```

Conviene recordar que, aunque la persona visitante no vea un campo `hidden`, según la normativa de protección de datos no se debe recabar información de índole personal sin que esta haya sido informada previamente.

file

El tipo `file` permite subir archivos desde el formulario. Si se quiere permitir agregar más de un archivo, se debe incluir el atributo `multiple`; en ese caso, el botón mostrará "Elegir archivos" en lugar de "Elegir archivo". Es recomendable incluir también la etiqueta `<label>` para comunicar mejor a las personas usuarias la acción que pueden realizar. Conviene tener en cuenta que, con HTML y JavaScript, no es posible completar por sí solos la tarea de subida de un archivo al servidor: es necesario contar con los permisos y la programación adecuados en dicho servidor.

```
<label for="adjuntos">Adjunta tus archivos:</label>
<input type="file" id="adjuntos" name="adjuntos" multiple>
```

El botón de envío

Un formulario necesita ser enviado, es decir, en algún momento la persona usuaria debe tomar la decisión de enviar los datos que ha introducido. La forma más habitual de hacerlo es mediante un botón, que se puede crear de varias formas:

```
<input type="submit" value="Enviar">
<button type="submit">Enviar</button>
```

textarea

El elemento `<textarea>` representa un control de edición multilínea de texto plano. Se pueden incluir los atributos `rows` y `cols` para definir, respectivamente, el número de filas y de columnas visibles en el control.

```
<label for="mensaje">Tu mensaje:</label>
<textarea id="mensaje" name="mensaje" rows="4" cols="40"></textarea>
```

input type="color"

De todos los tipos nuevos de `input`, este es uno de los que más ha tardado en tener implementación en los navegadores, pero afortunadamente ya lo soportan los navegadores más importantes. Muestra un selector visual de color:

```
<label for="colorfd">Elegir color de fondo</label>
<input id="colorfd" name="colorfd" type="color">
```

datalist

`<datalist>` es un elemento que permite crear una lista de sugerencias de selección para un `<input>`. Para vincular el `datalist` con el campo se usa el atributo `list`, cuyo valor debe coincidir con el `id` del `datalist`:

```
<form action="gestiona.php" method="get">
    <label for="dia">Elige tu día de la semana:</label>
    <input list="dias" name="dia" id="dia">
    <datalist id="dias">
        <option value="Lunes">
        <option value="Martes">
        <option value="Miércoles">
        <option value="Jueves">
        <option value="Viernes">
        <option value="Sábado">
        <option value="Domingo">
    </datalist>
    <input type="submit">
</form>
```

pattern

Cuando ninguno de los tipos de campo anteriores ayuda a ofrecer la mejor usabilidad, el atributo `pattern` puede resultar de ayuda: permite indicar una expresión regular que el valor del campo debe cumplir para considerarse válido. Escribir expresiones regulares no es sencillo, así que conviene buscar ejemplos ya elaborados en la Web.

```
<form action="gestiona_datos.php">
    <label for="dni">DNI:</label>
    <input type="text" id="dni" name="dni"
        pattern="(([X-Z]{1})([-]?)(\d{7})([-]?)([A-Z]{1}))|((\d{8})([-]?)([A-Z]{1}))"
```

```
        title="Debes incluir un DNI correcto con 8 números y una letra en mayúscula">
    <input type="submit">
</form>
```

En el ejemplo anterior, el atributo `pattern` contiene una expresión regular para validar el formato del NIF español, y el contenido del atributo `title` es el mensaje que el navegador muestra a la persona usuaria cuando la validación falla.

placeholder

`placeholder` se usa cuando un campo de formulario no tiene ningún valor. Cuando el cursor se coloca en el campo, el texto desaparece; si se saca el cursor y el campo sigue vacío, el texto vuelve a aparecer. No se trata, por tanto, de un valor que se pueda enviar: es simplemente una ayuda visual para que la persona usuaria entienda qué debe introducir en ese campo. Su aspecto tiene el color del texto normal algo atenuado, aunque se puede modificar mediante el pseudoelemento `::placeholder` de CSS.

```
<label for="cp">Código postal:</label>
<input type="text" id="cp" name="cp" placeholder="Incluye los 5 números">
```

autofocus

El atributo `autofocus` es un atributo *booleano*, lo que significa que no necesita declarar ningún valor: basta con su presencia. Cuando está presente, especifica que ese elemento `<input>` debe recibir el foco de forma automática en cuanto se carga la página.

required

El atributo `required`, en los navegadores modernos, impide que el formulario se envíe si el campo no ha sido rellenado. Confiando en este comportamiento del navegador, se puede prescindir de JavaScript para las validaciones más sencillas.

```
<form action="cambia.php" method="post" name="login">
    <label for="usuario">Usuario:</label>
    <input type="text" name="usuario" id="usuario" required>
    <label for="clave">Clave:</label>
    <input type="password" name="clave" id="clave" required>
    <input type="submit" value="Enviar">
</form>
```

autocomplete

La mayoría de los navegadores incorporan funciones adicionales pensadas para mejorar la facilidad de uso y la seguridad, como la cumplimentación automática de campos de formulario. La mayoría de las veces resulta muy útil, pero en ocasiones puede ser molesto o incluso peligroso, como en un campo destinado a introducir los datos de una tarjeta bancaria. El atributo `autocomplete="off"` permite desactivar este comportamiento:

```
<form action="gestiona_datos.php" method="get" autocomplete="off">
    <label for="tarjeta">Número de tarjeta:</label>
    <input type="text" id="tarjeta" name="tarjeta" autocomplete="off">
</form>
```

2.12. Herramientas de diseño web

Herramientas propietarias (Adobe XD, Sketch, Figma, InVision Studio) y libres (GIMP, Inkscape, Blender, Gravit Designer, Pencil) para el diseño de interfaces web.

Antes de escribir el código de una página, es habitual diseñar primero su aspecto visual con herramientas específicas de diseño. Existen tanto herramientas propietarias de pago como alternativas libres y gratuitas.

Herramientas propietarias

HERRAMIENTA	DESCRIPCIÓN
Logotipo de Adobe X Adobe XD	Herramienta de diseño y prototipado de interfaz de usuario desarrollada por Adobe. Ampliamente utilizada para diseñar interfaces de aplicaciones y sitios web, ofrece diseño de alta fidelidad, prototipado interactivo y colaboración en equipo.
Logotipo de Sketch Sketch	Herramienta de diseño exclusiva para macOS. Muy popular entre diseñadores de interfaces de usuario, se emplea para crear diseños de alta calidad, iconos y prototipos.
Logotipo de Figma Figma	Herramienta de diseño basada en la nube que permite a los equipos colaborar en tiempo real en proyectos de diseño de interfaces. Muy versátil, la utilizan tanto diseñadores como desarrolladores.

HERRAMIENTA	DESCRIPCIÓN
Logotipo de InVisior InVision Studio	Herramienta de diseño y prototipado que permite crear interacciones complejas y animaciones en las interfaces de usuario. Especialmente útil para quienes se centran en la animación y el prototipado.

Herramientas libres

HERRAMIENTA	DESCRIPCIÓN
Logotipo de GIMP GIMP (*GNU Image Manipulation Program*)	Potente herramienta de edición de imágenes de código abierto, utilizada para crear y editar elementos gráficos en interfaces de usuario. Es una alternativa libre a Adobe Photoshop.
Logotipo de Inkscape Inkscape	Editor de gráficos vectoriales de código abierto, útil para crear iconos, ilustraciones y elementos de interfaz de usuario en formato vectorial.
Logotipo de Blender Blender	Conocido principalmente como software de modelado y animación 3D, también se puede utilizar para crear elementos 3D en interfaces de usuario, como gráficos y animaciones.
Logotipo de Gravit Designer Gravit Designer	Herramienta de diseño vectorial de código abierto que funciona tanto en línea como fuera de línea. Adecuada para crear elementos de interfaz de usuario, con características similares a Adobe Illustrator.
Logotipo de Pencil Project Pencil	Aplicación de código abierto para crear prototipos de interfaces de usuario. Permite crear bocetos interactivos y prototipos de forma rápida.

2.13. Introducción a CSS: cascada, herencia y formas de inclusión

Qué es CSS, cómo funcionan la cascada y la herencia, y las tres formas de incluir estilos en un documento: en línea, en el documento y en un archivo externo.

Una vez creada la estructura HTML correcta de un documento, se puede pasar a crear los estilos necesarios para darle forma, es decir, para aplicarle su aspecto visual, mediante CSS (*Cascading Style Sheets*, hojas de estilo en cascada): un lenguaje de estilo utilizado para controlar la apariencia y presentación de los elementos de un documento HTML.

Herencia y cascada

CSS utiliza el concepto de herencia: los estilos se pueden aplicar a elementos secundarios si ya se han aplicado a sus elementos padres. Podemos hablar de herencia cuando se establece el valor de una propiedad en un elemento y, a partir de ese momento, los elementos situados en un nivel jerárquico inferior adquieren el valor de esa propiedad de forma automática.

Además, CSS sigue un modelo de cascada: los estilos se pueden combinar y anular en función de la especificidad del selector y del orden en que se declaran las reglas. Una misma página puede disponer de más de una hoja de estilo, ya que en su composición final confluyen varios tipos de hojas: la que aporta la persona que visualiza la página, la que proporciona el propio navegador, y la que define quien está diseñando la página web.

Esta confluencia de hojas de estilo debe seguir cierto orden de prioridad para que el resultado sea realmente operativo: ante un conjunto de reglas en conflicto, el navegador dará más peso a unas que a otras.

Formas de incluir estilos

Como sucede con JavaScript, se puede incluir CSS en un documento de tres formas posibles.

En línea

Se usa el atributo `style` directamente en el elemento sobre el que se quiere aplicar el estilo. Es el método más directo y sencillo de aplicar, pero también el menos recomendable, ya que mezcla contenido y presentación y es difícil de mantener en un proyecto grande.

```
<p style="color:orange;font-weight:bold">Texto destacado</p>
```

En el documento

Se usa la etiqueta `<style>`, que debe estar siempre dentro de `<head>`, al contrario de lo que sucede con `<script>`, que puede colocarse en cualquier parte del documento. Con este método hay que aplicar los estilos mediante selectores, para poder indicar a qué elemento o elementos se aplica cada regla. Este elemento puede llevar el atributo `type="text/css"`, aunque en la actualidad no es necesario, y el atributo `media`, que admite valores como `all` (todos), `print` (impresión) o `screen` (pantalla), para definir el medio al que están destinados los estilos.

```
<head>
    <style>
        p {
            color: orange;
        }
    </style>
</head>
```

En un documento externo

En este caso hay dos maneras posibles. La primera, mediante `<link>` y su atributo `href` (*hypertext reference*), que indica la ubicación del archivo:

```
<link rel="stylesheet" href="estilos.css">
```

La segunda, mediante la regla `@import` dentro de una hoja de estilos. Inicialmente, ambos métodos realizan la misma tarea (el primero desde HTML, el segundo desde CSS), pero el autor y divulgador Steve Souders aconseja usar `<link>` y evitar `@import`, por la clara diferencia de rendimiento entre ambas opciones.

```
@import url("estilos.css");
```

Orden de prioridad entre los tres métodos

Los estilos incluidos en el propio documento se sobreponen a los que provienen de un archivo externo, siempre que estén especificados con el mismo selector y el mismo valor; y los estilos en línea se sobreponen, a su vez, a los estilos incluidos en la página, en las mismas circunstancias. En general, las declaraciones de estilos aplicadas sobre elementos más específicos se sobreponen sobre las más genéricas.

2.14. Selectores CSS

Selectores básicos (tipo, clase, id, universal, atributo), selectores combinados y pseudoclases y pseudoelementos más habituales.

Los selectores se utilizan para elegir los elementos HTML a los que se aplicarán los estilos. Se puede seleccionar por etiqueta, clase, id u otros atributos. Recordemos la estructura de una regla CSS:

```
p {
    color: orange;
}
```

En este ejemplo, `p` es el selector, `color: orange;` es la declaración, `color` es la propiedad y `orange` es el valor.

Selectores básicos

Selector	Selecciona	Ejemplo
De tipo	Todos los elementos que coinciden con el nombre de etiqueta indicado.	`main { text-decoration: underline; }`
De clase	Todos los elementos con el atributo class indicado, precedido de un punto.	`.col { width: 25%; }`
De id	El único elemento con ese id, precedido de una almohadilla (#).	`#menusuperior { background-color: #eee; }`
Universal	Todos los elementos del documento, con el símbolo asterisco.	`* { color: yellow; }`
De atributo	Elementos según el nombre de un atributo, su valor, o ambos.	`[type] { border: 1px solid black; }`

Variantes del selector de atributo

SINTAXIS	SELECCIONA ELEMENTOS...
`[attr]`	que tienen ese atributo, sea cual sea su valor.
`[attr=value]`	cuyo atributo tiene exactamente ese valor.
`[attr~=value]`	cuyo atributo contiene, entre varios valores separados por espacios, al menos ese valor.
`[attr|=value]`	cuyo atributo es igual a ese valor, o comienza por él seguido de un guion.
`[attr^=value]`	cuyo atributo empieza por ese valor.
`[attr$=value]`	cuyo atributo termina con ese valor.
`[attr*=value]`	cuyo atributo contiene ese valor en cualquier posición.

Selectores combinados

- Hermanos adyacentes: el segundo elemento sigue directamente al primero y ambos comparten el mismo elemento padre. `section + aside { background-color: beige; }`
- Hijo: selecciona los elementos que son hijos directos del primer elemento. `section > article { background-color: beige; }`
- Descendiente: selecciona los elementos que son descendientes del primer elemento, a cualquier profundidad. `main aside { background-color: beige; }`
- En cadena: permite asignar las mismas propiedades a varios elementos a la vez, separándolos por comas. `section, aside { margin: auto; }`

También es posible combinar un selector de elemento con uno de clase o de id, como en `aside.lateral { background-color: beige; }`. En este caso, la clase solo afectará a los elementos `<aside>` que además tengan esa clase, y no a otros elementos con la misma clase.

Pseudoclases y pseudoelementos

Las pseudoclases de vínculos afectan al estado de los enlaces:

`a:link`	Afecta a los vínculos no visitados.
`a:visited`	Afecta a los vínculos ya visitados.
`a:hover`	Afecta al vínculo cuando el cursor está sobre él.
`a:active`	Afecta al vínculo mientras se está pulsando.

Entre los pseudoelementos más habituales están:

`:nth-child()`	Selecciona un elemento en la posición indicada entre paréntesis, dentro de una lista de elementos hermanos.
`:first-child`	Selecciona el primero de una lista de elementos.
`:last-child`	Selecciona el último de una lista de elementos.

2.15. Colores y unidades en CSS

Los distintos formatos para definir colores en CSS (RGB, RGBA, HSL, hexadecimal) y las unidades de medida relativas y absolutas.

Colores en formato funcional

En el modelo RGB, cuanto mayor es el valor de cada componente, más luz de ese color se añade. Esto significa que con los tres colores en su valor máximo se obtiene blanco, y en su valor mínimo, negro. Los colores se pueden definir en formato decimal (de 0 a 255) o en porcentaje:

- `rgb(rojo, verde, azul)`: define el color a través de sus tres componentes, de 0 a 255, o en porcentaje.
- `rgba(rojo, verde, azul, alfa)`: igual que el anterior, pero con un cuarto valor de transparencia (*alfa*), expresado de 0 (transparente) a 1 (opaco).
- `hsl(matiz, saturación, luminosidad)`: define el color mediante sus valores de matiz, saturación y luminosidad, en porcentaje o decimal.
- `hsla(matiz, saturación, luminosidad, alfa)`: igual que el anterior, añadiendo el canal alfa de transparencia.

El sistema hexadecimal

Puesto que nuestro sistema de numeración habitual es decimal y solo dispone de diez dígitos, para representar el sistema hexadecimal se adoptó la convención de usar las seis primeras letras del alfabeto latino para suplir los dígitos que faltan. El conjunto de símbolos es el siguiente:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
```

El modelo RGB hexadecimal

Aplicar este modelo para definir un color sigue estos pasos:

1. Determinar los porcentajes de rojo, verde y azul que participan en el color que se quiere obtener.
2. Transformar el valor de cada porcentaje al sistema hexadecimal, obteniendo un nuevo valor. Por lo general, los programas informáticos realizan este cambio de formato de manera automática.
3. Encadenar los tres valores resultantes, en el mismo orden RGB, para obtener un código con el color deseado, precedido siempre del símbolo `#`.

El color rojo puro, por ejemplo, se representa así: `#FF0000`, es decir, 255 de rojo y 0 de verde y de azul.

Una ventaja del formato hexadecimal es que se puede comprimir cuando sus tres componentes están formadas por pares de dígitos iguales. Así, el mismo rojo puro se puede representar también como `#f00`, que en el sistema decimal equivaldría igualmente a 255, 0, 0.

Unidades de medida

Para conseguir una mayor accesibilidad y usabilidad, es especialmente importante utilizar unidades relativas, sobre todo para el tamaño del texto.

UNIDADES RELATIVAS	DESCRIPCIÓN
`em`	Representa el alto de la letra M mayúscula de la fuente utilizada; es relativo al `font-size` del propio elemento.
`rem`	Igual que `em`, pero relativo al `font-size` del elemento raíz del documento.
`ex`	Representa el alto de la letra X de la fuente utilizada.
`%`	Porcentaje respecto al valor del elemento contenedor.

También se pueden usar palabras clave relativas: `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large` y `xx-large`.

UNIDADES ABSOLUTAS	DESCRIPCIÓN
`mm / cm`	Milímetros y centímetros.
`in`	Pulgadas (1 in = 2,54 cm).
`pt`	Puntos (1 pt = 1/72 in).
`pc`	Picas (1 pc = 12 pt).
`px`	Píxeles.

2.16. El modelo de caja: márgenes, bordes y relleno

Las propiedades que definen el tamaño y el espaciado de cualquier elemento: width, height, margin, padding y las distintas propiedades del borde.

Todos los elementos que componen una página web pueden representarse mediante cajas rectangulares (el llamado modelo de caja de CSS). Conocer las propiedades que controlan el tamaño y el espaciado de esas cajas es imprescindible para maquetar cualquier diseño.



Rectángular con los cuatro lados numerados: 1 superior, 2 derecho, 3 inferior y 4 izquierdo

Ancho y alto

`width`	Establece el ancho de una caja o de otros elementos, como las imágenes.
`height`	Establece el alto de una caja o de otros elementos.
`min-width` `max-width`	Establecen el ancho mínimo y máximo de la caja.
`min-height` `max-height`	Establecen el alto mínimo y máximo de la caja.

Estas propiedades admiten como valor una longitud (en las unidades ya vistas), un porcentaje (calculado respecto a la medida del bloque que contiene la caja), o `auto`, que calcula el valor automáticamente.

Márgenes

Las propiedades `margin-top`, `margin-right`, `margin-bottom` y `margin-left` aplican, respectivamente, márgenes superior, derecho, inferior e izquierdo a un elemento de tipo caja (aunque también las admiten las imágenes). Los elementos en línea solo admiten margen izquierdo y derecho. Estas propiedades admiten valores negativos, como por ejemplo `margin-left: -20px;`.

`margin` es la propiedad resumida que permite establecer los cuatro márgenes a la vez, en el orden superior, derecho, inferior e izquierdo:

```
.caja {
    margin: 10px 20px 10px 20px;
}
```

Rellenos (padding)

De forma equivalente a los márgenes, las propiedades `padding-top`, `padding-right`, `padding-bottom` y `padding-left` aplican, respectivamente, relleno superior, derecho, inferior e izquierdo a un elemento caja. La diferencia esencial con el margen es que el relleno forma parte del interior de la caja (entre el borde y el contenido), mientras que el margen queda fuera del borde. `padding` es su propiedad resumida.

Bordes

Las propiedades de borde siguen el mismo patrón que márgenes y rellenos, con una variante para cada uno de los cuatro lados de la caja:

`border-width` (grosor)	Admite los valores `thin` (fino), `medium` (medio), `thick` (grueso), o una longitud expresada en las unidades conocidas. No admite valores negativos.
`border-color` (color)	Admite cualquier valor de color, o `transparent`, que hace el borde invisible aunque conserve su grosor.
`border-style` (estilo)	Admite valores como `none` (sin borde), `hidden` (oculto), `dotted` (punteado), `dashed` (discontinuo), `solid` (continuo), `double` (doble línea), `groove` (efecto de hendidura), `ridge` (efecto convexo), `inset` (el elemento parece incrustado) y `outset` (el elemento parece resaltado).

`border` es la propiedad resumida que combina grosor, estilo y color de los cuatro lados a la vez:

```
.caja {
    border: 2px solid #333;
}
```

2.17. Posicionamiento, z-index y flexbox

La propiedad position y sus valores, el control de capas con z-index, y una introducción al modelo de caja flexible con display: flex.

La propiedad display

`display` establece el tipo de caja de un elemento, es decir, cómo se dispone respecto al resto de elementos del documento:

`block`	Convierte al elemento en un bloque.
`inline`	Elemento en línea. Solo permite margen y relleno laterales.
`inline-block`	Se comporta como un elemento en línea, pero admite margen y relleno en todos los lados.
`list-item`	Genera una caja de bloque y una caja de elemento de lista.
`none`	El elemento desaparece por completo, como si no existiera en el documento.

`flex`	Convierte al elemento en un contenedor flexible (lo veremos a continuación).

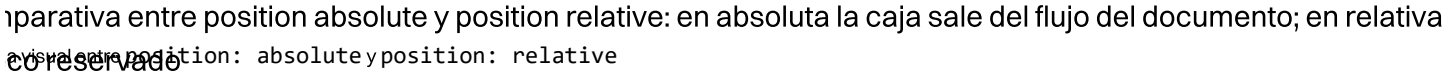
`table`
`inline-table`
`table-row`
`table-cell` , etc.
	Aplican comportamientos propios de una tabla a elementos que no lo son.

Posicionamiento

La propiedad `position`, junto con `top`, `right`, `bottom` y `left`, permite controlar la posición que ocupan las cajas y la relación que se establece entre ellas:

`static` (valor por defecto)	Posicionamiento normal. Con este valor, las propiedades `top`, `right`, `bottom` y `left` no tienen efecto.
`absolute`	Saca al elemento del flujo normal del documento, pasando a tener coordenadas absolutas controladas por `top`, `right`, `bottom` y `left`, tomando como referencia su elemento contenedor posicionado más cercano.
`relative`	También saca al elemento del flujo del documento en cuanto a su desplazamiento visual, pero el hueco que deja lo sigue reservando. Su desplazamiento se controla con las mismas cuatro propiedades, tomando como referencia su posición inicial.
`fixed`	Se comporta como `absolute`, pero queda inmóvil en la pantalla al hacer *scroll*. No lo reconocen algunos navegadores muy antiguos, como Internet Explorer 6.

Las propiedades `top`, `right`, `bottom` y `left` admiten como valor una longitud, un porcentaje (calculado respecto al elemento que las contiene) o `auto`.



a visual entre `position: absolute` y `position: relative`

z-index

`z-index` especifica el orden de visualización cuando varios elementos se solapan, estableciendo niveles o capas de abajo hacia arriba. Su valor es un número: cuanto mayor es, más arriba se sitúa la capa correspondiente; también admite el valor `auto`.

```
#tierra {
    position: absolute; width: 300px; top: 20px; left: 30px; z-index: 1;
}
```

```
#platillo {
    position: absolute; width: 60px; top: 150px; left: 100px; z-index: 2;
}
```

ración de un platillo volante situado sobre un planeta Tierra, resultado del ejemplo de código con position y z-in
tro cajas superpuestas mostrando cómo z-index determina cuál se dibuja más arriba

El modelo de caja flexible (flexbox)

En el modelo flexible, los elementos hijos de un contenedor se pueden distribuir en dirección vertical u horizontal, adaptando su tamaño para rellenar o encogerse dentro de su elemento "padre". Los elementos flexibles se pueden estirar para aprovechar el espacio disponible, en proporción a su factor de crecimiento, o encogerse para evitar el desbordamiento, según su factor de contracción.

Basta con añadir a un contenedor la propiedad `display: flex` para que los elementos que contiene se conviertan en flexibles:

```
.contenedor {
    display: flex;
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
    gap: 1rem;
}
```

En este ejemplo se combinan varias propiedades y valores para que las cajas del contenedor se distribuyan en fila (`row`), separadas uniformemente (`space-between`), centradas verticalmente (`center`) y con un espacio fijo entre ellas (`gap`).

2.18. Fondos, imágenes y filtros en CSS

Las propiedades background para controlar imágenes de fondo, el uso de varias imágenes de fondo a la vez y los efectos visuales de la propiedad filter.

Fondos

Los fondos y las imágenes de fondo son un recurso estético muy usado en el diseño de páginas web. En este caso, las imágenes no se insertan como contenido (con `<img>`), sino mediante CSS, como parte decorativa de un elemento.

PROPIEDAD	DESCRIPCIÓN Y VALORES
`background-image`	Establece una imagen de fondo. Valores: `none` (por defecto), `url(imagen.jpg)`.
`background-color`	Establece el color de fondo de un elemento. Valores: `transparent` (por defecto), o cualquier color.
`background-repeat`	Determina cómo se repite la imagen de fondo. Valores: `repeat`, `repeat-x`, `repeat-y`, `no-repeat`.
`background-position`	Determina dónde se coloca la imagen. Valores: `bottom`, `center`, `left`, `right`, un porcentaje, o una longitud calculada desde la esquina superior izquierda del elemento (por ejemplo `0 10px`).
`background-origin`	Determina si el fondo se posiciona respecto al borde, al relleno o al margen. Valores: `border-box`, `padding-box`, `margin-box`.
`background-clip`	Declara dónde se recorta la imagen de fondo. Valores: `border-box`, `padding-box`, `margin-box`.
`background-attachment`	Determina si la imagen de fondo permanece estática o se desplaza junto con el contenido. Valores: `scroll`, `fixed`.

`background` es la propiedad resumida que combina las anteriores: `background: color url() repeat position;`

background-size

CSS3 incorporó nuevas propiedades para las imágenes de fondo, entre las que destaca `background-size`. Además del tamaño horizontal y vertical, admite valores como `cover` (agranda la imagen para cubrir todo el elemento, recortando si es necesario) o `contain` (ajusta la imagen para que se vea completa dentro del elemento).

```
.cabecera {
    background-image: url("fondo.jpg");
    background-size: cover;
    background-position: center;
    background-repeat: no-repeat;
}
```

imagen.jpg con una ilustración floral de fondo ajustada mediante background-size

Varias imágenes de fondo

A partir de CSS3 es posible incluir más de una imagen de fondo en un mismo elemento, separando cada una por una coma; la primera imagen de la lista se dibuja más cerca de la persona que observa la pantalla, y el resto se van apilando detrás:

```
.elemento {
    background-image: url("estrellas.png"), url("nubes.jpg");
    background-position: left top, right bottom;
    background-repeat: no-repeat, no-repeat;
}
```

Filtros

La propiedad `filter` permite aplicar efectos visuales a los elementos, como un desenfoque, un cambio de saturación o un efecto sepia:

```
filter: blur(5px);
filter: brightness(0.4);
filter: contrast(200%);
filter: drop-shadow(16px 16px 20px blue);
filter: grayscale(50%);
filter: hue-rotate(90deg);
filter: invert(75%);
filter: opacity(25%);
filter: saturate(30%);
filter: sepia(60%);
```

El efecto que se consigue con el filtro `opacity` también se puede lograr mediante la propiedad CSS del mismo nombre, `opacity`, aplicada directamente sobre el elemento.

2.19. Diseño responsivo: mobile first, viewport y densidad de pantalla

Por qué diseñar primero para móvil, el papel del viewport, y qué es la densidad de píxeles y cómo afecta a la forma en que se ve un diseño en distintos dispositivos.

Ya hemos visto qué es el enfoque Mobile First. Veamos ahora por qué tiene tanto sentido en la actualidad, y qué conceptos técnicos hay detrás de la adaptación de un diseño a distintos dispositivos.

¿Por qué crear páginas pensando primero en el móvil?

Según la Encuesta sobre Equipamiento y Uso de Tecnologías de Información y Comunicación en los Hogares, realizada por el Instituto Nacional de Estadística (INE), el 99,5 % de los hogares españoles disponía en 2021 de un teléfono móvil. Otras fuentes periodísticas aseguran que en torno al 91,5 % de las personas internautas acceden a la Web a través de un teléfono móvil. Aunque estas cifras no significan que esas personas naveguen exclusivamente desde un smartphone, son un argumento contundente para priorizar el diseño móvil.

El viewport, otra vez

Como ya vimos, la ventana gráfica o *viewport* es el área visible de la página web, y su tamaño varía según el dispositivo. La etiqueta `<meta name="viewport">` es el punto de partida imprescindible de cualquier diseño responsivo, ya que sin ella el navegador móvil trataría la página como si fuera de escritorio.

Densidad de pantalla

Cuando hablamos del tamaño de un televisor, un monitor o un teléfono móvil, solemos hacerlo en pulgadas (una pulgada equivale a 2,54 cm), midiendo siempre la diagonal del dispositivo. Es habitual ver anuncios que hablan de "pantallas de 60 pulgadas" para un televisor, o de que un modelo de smartphone tiene "6,7 pulgadas".

Un píxel es un punto que emite un único matiz de color en pantalla. La densidad de píxeles es la cantidad de puntos que caben en una pulgada de pantalla: cuantos más píxeles haya por unidad de superficie, más nítida se verá la imagen. Se suele hablar de ppp (píxeles por pulgada), o en inglés ppi (*pixels per inch*) o dpi (*dots per inch*).

La densidad de pantalla se calcula con la siguiente fórmula:

```
DP = AP en píxeles / AP en pulgadas

DP = densidad de pantalla
AP = anchura de la pantalla
```

Por ejemplo, un smartphone Samsung Galaxy A41 tiene una pantalla de 6,1 pulgadas de diagonal, con una anchura de pantalla de 2,5 pulgadas y 921 píxeles. Aplicando la fórmula:

```
921 / 2,5 = 368 ppp
```

Se considera que una pantalla tiene alta densidad cuando su densidad supera los 160 ppp, algo que en la actualidad ocurre en todos los teléfonos móviles. Esto tiene una consecuencia práctica importante: la pantalla de un móvil, al tener mayor densidad de píxeles por pulgada, haría que una letra de 16 píxeles se viera físicamente más pequeña en un smartphone que en un ordenador de sobremesa. Para corregir este desajuste, tanto Android como iOS aplican sus propios mecanismos de compensación: dp (píxeles de densidad independiente) en Android, y los factores de escala @1x, @2x y @3x en iOS.

Tipos de medios en CSS

Además de adaptarse al tamaño de pantalla, CSS permite definir estilos distintos según el tipo de medio en el que se va a mostrar el documento:

MEDIO	DESCRIPCIÓN
`braille`	Dispositivos táctiles braille para personas ciegas o con visión reducida.
`embossed`	Impresoras braille.
`handheld`	Antiguos dispositivos PDA.
`print`	Impresión del documento.
`projection`	Proyección o presentaciones tipo diapositivas.
`screen`	Pantalla digital.
`speech`	Lectores de páginas (síntesis de voz).
`tty`	Obsoleto: terminales de caracteres de ancho fijo, como teletipos y consolas.
`tv`	Televisores.

En los últimos años, los medios más usados con diferencia han sido `screen` y `print` ("pantalla" e "impresión"), aunque incluso este último ha caído en desuso, gracias a que los navegadores actuales convierten por sí solos los documentos web en archivos imprimibles con un resultado razonable.

2.20. Puntos de ruptura, media queries y CSS Grid

Cómo definir puntos de ruptura con media queries para adaptar el diseño a distintos anchos de pantalla, y una introducción a la maquetación con CSS Grid.

Puntos de ruptura (breakpoints)

Los puntos de ruptura (*break points*) son los valores de ancho (o, más raramente, de alto) que quien diseña una página elige para determinar qué diseño se debe aplicar en cada caso, según el tamaño de pantalla. De esta manera se puede aplicar una regla específica a un elemento, o a toda la página, únicamente cuando se cumple una condición de tamaño. Aunque este mecanismo, las *media queries*, existe desde hace tiempo, hoy se usa sobre todo para definir estos puntos de ruptura.

Los puntos de ruptura no deben elegirse de forma aleatoria ni a la ligera: deben responder a los tamaños de pantalla reales que utilizan las personas usuarias en la actualidad, y por supuesto al propio diseño visual que se quiera mostrar. Aun así, existen algunos puntos de ruptura que se han convertido en estándares habituales, como 480px, 768px, 1024px y 1200px, correspondientes aproximadamente a móviles, tablets, portátiles y pantallas de escritorio.

En el siguiente ejemplo, los párrafos tienen un tamaño de fuente inicial de 20 píxeles, que aumenta a 22 cuando la pantalla mide 1024 píxeles de ancho o menos, y vuelve a aumentar a 24 cuando la pantalla mide 480 píxeles de ancho o menos:

```
p {
    font-size: 20px;
}

@media (max-width: 1024px) {
    p {
        font-size: 22px;
    }
}

@media (max-width: 480px) {
    p {
        font-size: 24px;
    }
}
```

CSS Grid

Aunque CSS Grid es una técnica relativamente reciente, ya es compatible con la mayoría de los navegadores modernos, y su uso crece con rapidez. Es una herramienta muy útil para quienes desarrollan páginas web y buscan una forma más flexible y potente de crear diseños. Con CSS Grid se puede crear una cuadrícula o matriz de filas y columnas en la que los elementos de la página se colocan y alinean de forma muy precisa.

Veamos un ejemplo con seis elementos distribuidos en una cuadrícula de dos columnas:

```
<div class="contenedor">
    <div class="e e1">Elemento 1</div>
```

```
    <div class="e e2">Elemento 2</div>
    <div class="e e3">Elemento 3</div>
    <div class="e e4">Elemento 4</div>
    <div class="e e5">Elemento 5</div>
    <div class="e e6">Elemento 6</div>
</div>
```

```
.contenedor {
    display: grid;
    grid-template-columns: 1fr 1fr;
    grid-template-rows: 50px 50px 50px;
    grid-gap: 10px 10px;
}
.e {
    background-color: greenyellow;
    border: 2px solid green;
    border-radius: 10px;
    padding: 10px;
}
.e1 { grid-column: 1 / 2; grid-row: 1 / 2; }
.e2 { grid-column: 2 / 3; grid-row: 1 / 2; }
.e3 { grid-column: 1 / 2; grid-row: 2 / 3; }
.e4 { grid-column: 2 / 3; grid-row: 2 / 3; }
.e5 { grid-column: 1 / 3; grid-row: 3 / 4; }
.e6 { grid-column: 1 / 3; grid-row: 4 / 5; }
```

Con `display: grid` convertimos el contenedor en una cuadrícula. `grid-template-columns` y `grid-template-rows` definen el número y el tamaño de columnas y filas, mientras que `grid-gap` establece la separación entre celdas. Después, cada elemento se posiciona indicando en qué columnas y filas de la cuadrícula debe situarse, mediante `grid-column` y `grid-row`.

2.21. Validación de documentos HTML y CSS

Para qué sirve validar un documento HTML o CSS, qué herramientas ofrece el W3C para hacerlo y los errores más habituales que detectan.

Escribir código HTML o CSS que "se ve bien" en el navegador no significa necesariamente que ese código sea correcto. Los navegadores son muy tolerantes con los errores: intentan interpretar el documento aunque falte una etiqueta de cierre, aunque un atributo no lleve comillas, o aunque se use una propiedad CSS mal escrita, simplemente ignorando lo que no entienden. Esa tolerancia es cómoda, pero también esconde errores que pueden mostrarse de forma distinta según el navegador, el dispositivo o una futura actualización.

¿Qué significa validar un documento?

Validar un documento HTML o CSS consiste en comprobar que cumple estrictamente las reglas de sintaxis definidas por el estándar correspondiente (una DTD o esquema, en el caso de HTML; la gramática de CSS, en el caso de las hojas de estilo). El resultado de la validación es un informe con los errores y avisos encontrados, indicando la línea exacta del documento en la que se producen.

Por qué conviene validar

- Consistencia entre navegadores: un documento bien formado tiene muchas más probabilidades de mostrarse igual en Chrome, Firefox, Safari o Edge, ya que deja menos margen para que cada navegador "interprete a su manera" un error.
- Detección temprana de errores: una etiqueta sin cerrar, un atributo duplicado o un identificador repetido pueden pasar desapercibidos visualmente durante mucho tiempo, hasta que provocan un fallo difícil de localizar.
- Accesibilidad: muchos validadores señalan también problemas que afectan a las personas que usan lectores de pantalla, como imágenes sin atributo `alt` o formularios sin `<label>`.
- Mantenimiento: un código correcto y ordenado es más fácil de entender, ampliar y depurar en el futuro, ya sea por la misma persona o por otra distinta.

Herramientas del W3C

El propio W3C, el organismo que define los estándares web, ofrece validadores gratuitos y de uso libre:

- Nu Html Checker (`validator.w3.org/nu`), sucesor del clásico *W3C Markup Validation Service*: valida documentos HTML5.
- CSS Validation Service (`jigsaw.w3.org/css-validator`): valida hojas de estilo CSS.

Ambos servicios admiten tres formas de validar un documento:

1. Por URL: se indica la dirección de una página ya publicada.
2. Subiendo un archivo: se selecciona el archivo HTML o CSS desde el propio equipo.
3. Pegando el código directamente: útil para comprobar un fragmento antes de publicarlo.

Errores habituales que detecta un validador

- Etiquetas abiertas que no se han cerrado, o cerradas en el orden incorrecto (anidamiento incorrecto).
- Atributos obligatorios ausentes, como `alt` en `<img>`.
- Valores de `id` duplicados dentro del mismo documento.
- Elementos de bloque colocados dentro de elementos en línea, lo que no está permitido.

- Ausencia del `<!DOCTYPE html>`, que hace que el navegador interprete la página en "modo peculiar" (*quirks mode*) en lugar de en modo estándar.
- Propiedades o valores CSS mal escritos, o no soportados por ninguna especificación.

Conviene aclarar que un documento puede "verse bien" en el navegador y, aun así, contener varios errores de validación: el navegador los está corrigiendo silenciosamente por su cuenta. Por eso la validación es una buena práctica que se recomienda incorporar como parte habitual del proceso de desarrollo, y no solo como un paso final antes de publicar.

2.22. Lenguajes de marcas para la sindicación de contenidos

Qué es la sindicación de contenidos y cómo se estructuran los documentos RSS y Atom que permiten distribuir y suscribirse a las novedades de un sitio web.

La sindicación de contenidos es el mecanismo que permite a un sitio web publicar un resumen de sus novedades (artículos de un blog, noticias, episodios de un pódcast...) en un formato estandarizado, para que otras aplicaciones o sitios web puedan mostrar, reutilizar o suscribirse automáticamente a esas novedades sin tener que visitar la página original. Los lenguajes de marcas más utilizados para este fin son RSS y Atom, ambos basados en XML.

RSS (Really Simple Syndication)

Un documento RSS es, en esencia, un archivo XML con una estructura muy concreta: un elemento raíz `<rss>`, que contiene un único `<channel>` (el propio sitio o canal), y dentro de él, tantos elementos `<item>` como novedades se quieran publicar:

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0">
    <channel>
        <title>Blog de Accedia</title>
        <link>https://www.accedia.example/blog</link>
        <description>Últimas noticias del curso</description>

        <item>
            <title>Nueva unidad de HTML y CSS</title>
            <link>https://www.accedia.example/blog/nueva-unidad</link>
            <description>Ya está disponible la UD2 del módulo LMSGI.</description>
            <pubDate>Sat, 13 Sep 2025 10:00:00 GMT</pubDate>
        </item>
    </channel>
</rss>
```

Cada `<item>` suele incluir, como mínimo, un título (`<title>`), un enlace a la publicación completa (`<link>`), un resumen o descripción (`<description>`) y la fecha de publicación (`<pubDate>`).

Atom

Atom es un formato más moderno que RSS, creado para resolver algunas de sus ambigüedades (por ejemplo, RSS no obliga a que cada elemento tenga un identificador único, algo que Atom sí exige). Su elemento raíz es `<feed>`, y cada novedad se representa con un elemento `<entry>`:

```
<?xml version="1.0" encoding="UTF-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
    <title>Blog de Accedia</title>
    <link href="https://www.accedia.example/blog"/>
    <updated>2025-09-13T10:00:00Z</updated>
    <id>urn:uuid:60a76c80-d399-11d9-b93C-0003939e0af6</id>

    <entry>
        <title>Nueva unidad de HTML y CSS</title>
        <link href="https://www.accedia.example/blog/nueva-unidad"/>
        <id>urn:uuid:1225c695-cfb8-4ebb-aaaa-80da344efa6a</id>
        <updated>2025-09-13T10:00:00Z</updated>
        <summary>Ya está disponible la UD2 del módulo LMSGI.</summary>
    </entry>
</feed>
```

Diferencias principales entre RSS y Atom

Aspecto ⟷	RSS	Atom
Elemento raíz ⟷	`<rss>`	`<feed>`
Cada novedad ⟷	`<item>`	`<entry>`
Identificador único ⟷	Opcional (`<guid>`)	Obligatorio (`<id>`)
Espacio de nombres XML ⟷	No obligatorio	Obligatorio (`xmlns`)

Aspecto ⟷	RSS	Atom
Especificación ⟷	Menos estricta, varias versiones (0.9x, 2.0)	Estándar único (RFC 4287)

Autodescubrimiento del feed

Para que un navegador o un lector de noticias pueda encontrar automáticamente el canal de sindicación de una página, es habitual declararlo en el `<head>` del documento HTML mediante una etiqueta `<link>`:

```
<link rel="alternate" type="application/rss+xml" title="Blog de Accedia" href="/feed.xml">
```

Gracias a esta declaración, cualquier aplicación compatible (lectores de RSS, agregadores de noticias, algunos navegadores) puede detectar el feed y ofrecer a la persona usuaria la opción de suscribirse a las novedades del sitio sin necesidad de visitarlo de forma manual.